

Building ML web apps with Gradio

---

# Gradio를 활용한 ML 웹 앱 제작

02 데이터와 모델 연동

## 목차 02 데이터와 모델 연동

---

1. 데이터 입력과 출력 처리
2. 다양한 입력 유형 다루기
3. 학습된 모델 통합

# 1. 데이터 입력과 출력 처리





- Interface, Block 등으로 데이터 입력과 출력을 처리할 수 있으며, 여러 형식을 지원
- 입, 출력 데이터 지원 형식 : 텍스트, 이미지, 오디오, 라벨 등

입력 데이터



name

ClearSubmit

출력 데이터



output

Flag



- 전처리 과정을 통해 입력 데이터를 함수 입력값 형식에 맞게 변환
- 후처리 과정을 통해 함수의 결과를 출력 데이터 형식에 맞게 변환

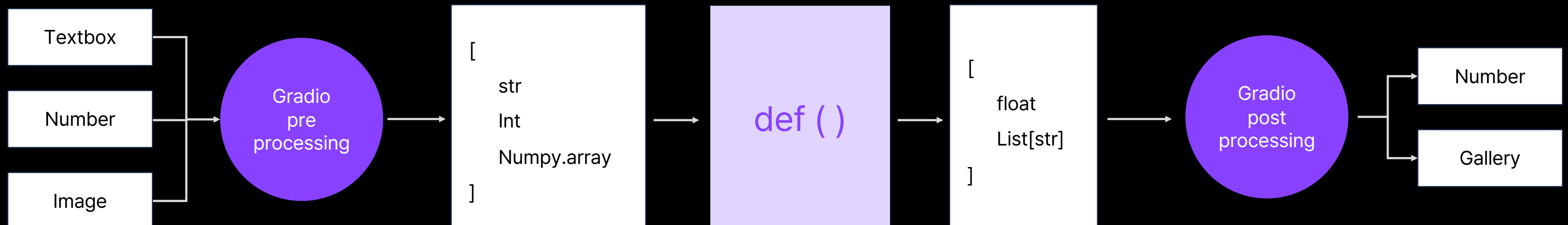
Input Components

Parameters

Your Function

Return Value(s)

Ouput Components





## • 전처리 예시

이미지를 PIL Image 객체로 변환, 크기를 100, 100로 변경

```
img = gr.Image(shape=(100, 100), type="pil")
```

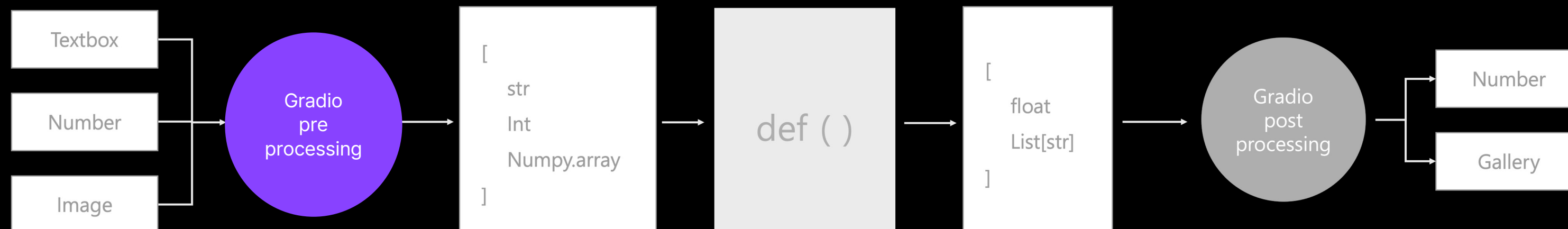
Input Components

Parameters

Your Function

Return Value(s)

Ouput Components



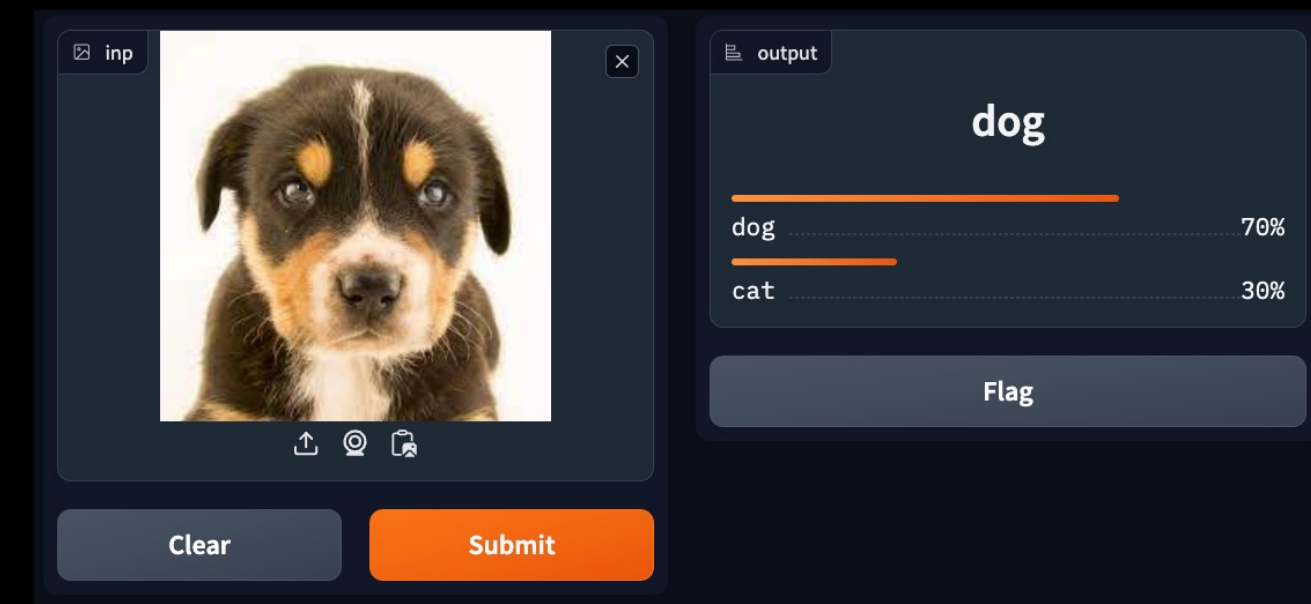
## • 후처리 예시

소프트맥스 결과값을 후처리하여 label의 형식으로 결과를 보여줌

```
import gradio as gr

def image_classifier(inp):
    return {'cat': 0.3, 'dog': 0.7}

demo = gr.Interface(fn=image_classifier, inputs="image", outputs="label")
demo.launch()
```



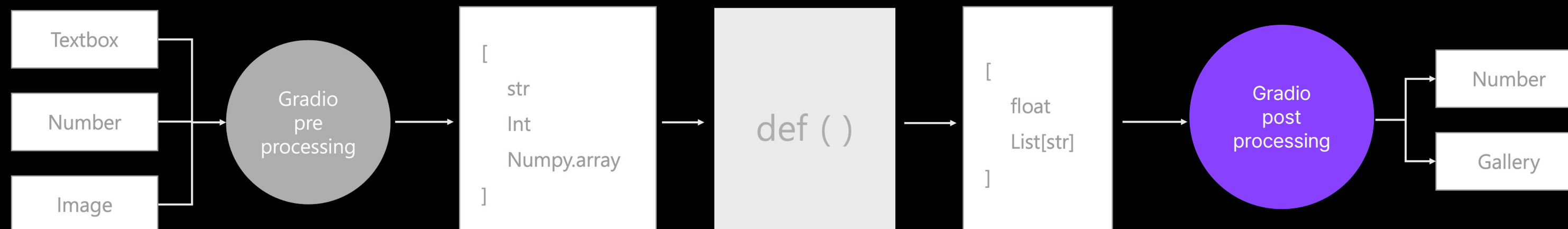
Input Components

Parameters

Your Function

Return Value(s)

Ouput Components





## 2. 다양한 입력 유형 다루기





# 1. 텍스트

Gradio를 활용한  
ML 웹 앱 제작

02 데이터와 모델 연동

2. 다양한 입력 유형  
다루기

## 입력 : 텍스트 / 출력 : 텍스트

```
import gradio as gr

def greet(name):
    return "Hello " + name + "!"

demo = gr.Interface(
    fn=greet,
    inputs=gr.Textbox(lines=2, placeholder="Name Here..."),
    outputs="text",
)
demo.launch()
```

1) 입력



2) 출력



name

Name Here...

Clear

Submit

output

Flag



## 2. 이미지

Gradio를 활용한  
ML 웹 앱 제작

02 데이터와 모델 연동

2. 다양한 입력 유형  
다루기

### 입력 : 이미지 / 출력 : 이미지

```
import numpy as np
import gradio as gr

def sepia(input_img):
    sepia_filter = np.array([
        [0.393, 0.769, 0.189],
        [0.349, 0.686, 0.168],
        [0.272, 0.534, 0.131]
    ])
    sepia_img = input_img.dot(sepia_filter.T)
    sepia_img /= sepia_img.max()
    return sepia_img

demo = gr.Interface(sepia, gr.Image(), "image")
demo.launch()
```

1) 입력

2) 출력

input\_img

  
이미지를 끌어 놓으세요

Clear

Submit

output



Flag



## 3. 오디오

Gradio를 활용한  
ML 웹 앱 제작

02 데이터와 모델 연동

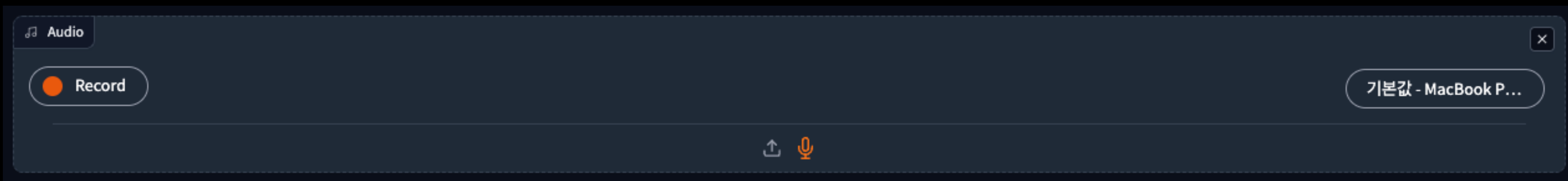
2. 다양한 입력 유형  
다루기

### 입력 : 오디오

```
import gradio as gr

with gr.Blocks() as demo:
    gr.Audio() → 1) 입력

demo.launch()
```







## 4. 멀티 입력, 출력

Gradio를 활용한  
ML 웹 앱 제작

02 데이터와 모델 연동

2. 다양한 입력 유형  
다루기

### 입력 : 텍스트, 체크박스, 슬라이더 / 출력 : 텍스트, 숫자

```
import gradio as gr

def greet(name, is_morning, temperature):
    salutation = "Good morning" if is_morning else "Good evening"
    greeting = f"{salutation} {name}. It is {temperature} degrees today"
    celsius = (temperature - 32) * 5 / 9
    return greeting, round(celsius, 2)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "checkbox", gr.Slider(0, 100)],
    outputs=["text", "number"],
)
demo.launch()
```

1) 입력

2) 출력

name

☐ is\_morning

temperature

Clear

Submit

output 0

output 1

Flag

## 입력 : 표 / 출력 : 표

matrix

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|----|
| 1 | ▲ | 2 | ▲ | 3 | ▲ | 4 | ▲ | 5 | ▲ | 6 | ▲ | 7 | ▲ | 8 | ▲ | 9 | ▲ | 10 |
| 2 |   | 6 |   | 0 |   | 4 |   | 5 |   | 5 |   | 4 |   | 4 |   | 9 |   | 4  |
| 1 |   | 1 |   | 0 |   | 6 |   | 2 |   | 2 |   | 5 |   | 2 |   | 6 |   | 1  |
| 3 |   | 1 |   | 2 |   | 4 |   | 4 |   | 6 |   | 0 |   | 9 |   | 8 |   | 2  |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |

↓ New row

→ New column

Clear

Submit

output

|   |  |   |   |  |   |   |  |   |   |  |   |   |  |   |
|---|--|---|---|--|---|---|--|---|---|--|---|---|--|---|
| 1 |  | ▲ | 2 |  | ▲ | 3 |  | ▲ | 4 |  | ▲ | 5 |  | ▲ |
| 2 |  |   | 1 |  |   | 3 |  |   |   |  |   |   |  |   |
| 6 |  |   | 1 |  |   | 1 |  |   |   |  |   |   |  |   |
| 0 |  |   | 0 |  |   | 2 |  |   |   |  |   |   |  |   |
| 4 |  |   | 6 |  |   | 4 |  |   |   |  |   |   |  |   |
| 5 |  |   | 2 |  |   | 4 |  |   |   |  |   |   |  |   |
| 5 |  |   | 2 |  |   | 6 |  |   |   |  |   |   |  |   |
| 4 |  |   | 5 |  |   | 0 |  |   |   |  |   |   |  |   |
| 4 |  |   | 2 |  |   | 9 |  |   |   |  |   |   |  |   |
| 9 |  |   | 6 |  |   | 8 |  |   |   |  |   |   |  |   |
| 4 |  |   | 1 |  |   | 2 |  |   |   |  |   |   |  |   |

Flag

```
import numpy as np

def transpose(matrix):
    return matrix.T

demo = gr.Interface(
    transpose,
    gr.Dataframe(type="numpy", datatype="number", row_count=5, col_count=3),
    "numpy",
    examples=[
        [np.zeros((3, 3)).tolist()],
        [np.ones((2, 2)).tolist()],
        [np.random.randint(0, 10, (3, 10)).tolist()],
        [np.random.randint(0, 10, (10, 3)).tolist()],
        [np.random.randint(0, 10, (10, 10)).tolist()],
    ],
    cache_examples=False
)

demo.launch()
```

1) 입력, 출력  
↑

### 3. 학습된 모델 통합





## 미리 학습된 모델을 불러와 사용

### Iris 꽃의 품종 예측

꽃잎, 꽃받침의 길이와 너비 값을 입력하면, 꽃의 세부 품종을 예측해줍니다.

Input Data

| SepalLengthCm ▲ | SepalWidthCm ▲ | PetalLengthCm ▲ | Pe |
|-----------------|----------------|-----------------|----|
| 4.9             | 3.6            | 1.4             | 0. |
| 6.1             | 2.9            | 4.7             | 1. |
| 6.3             | 3.4            | 5.6             | 2. |
| 6.2             | 2.2            | 4.5             | 1. |
| 6               | 3              | 4.8             | 1. |
| 5.1             | 3.8            | 1.6             | 0. |
| 5               | 3.6            | 1.4             | 0. |
| 6.2             | 2.9            | 4.3             | 1. |
| 6.7             | 3.1            | 4.7             | 1. |
| 6               | 2.7            | 5.1             | 1. |

↓ New row

→ New column

Clear

Submit

Predictions

| Species ▲ |
|-----------|
| 0         |
| 1         |
| 2         |
| 1         |
| 2         |
| 0         |
| 0         |
| 1         |
| 1         |
| 2         |

Flag